

**CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
CHANDUBHAI S PATEL INSTITUTE OF TECHNOLOGY**

**DEPARTMENT OF INFORMATION
TECHNOLOGY**

Lab Manual

**Dec-Jun 2019
(A.Y. 2018-19)**

IT247 Data Structures and Algorithms

Semester: 4th IT

**Department of
Information Technology**

Prepared by:

Mrs. Purvi Prajapati
Assistant Professor
IT Department
CSPIT, CHARUSAT

Mr. Amit Parmar
Assistant Professor
IT Department
CSPIT, CHARUSAT

Dr. Amit Thakkar
Associate Professor
IT Department
CSPIT, CHARUSAT

Credit and Hours

Teaching Scheme	Theory	Practical	Total	Credit
Hours/week	4	2	6	5
Marks	100	50	150	

Objective of the Course:

This course will introduce some of the most fundamental concepts in Computer Science like how data is represented and manipulated in computer systems in the form of stacks, queues, linked lists, tree, graph etc. To provide an in-depth knowledge in problem solving techniques and data structures.

The main objective to give the course is:

- To familiarize students with basic data structures and their use in fundamental algorithms.
- To teach the students how to select and design data structures and algorithms for a specified problem.
- To teach the students how data will be stored efficiently within computer memory.
- To select appropriate data structure and algorithm for a specified application.

Department of
Information Technology

Week - 1: Array

AIM:

- To implement Array.

OBJECTIVES:

- To understand memory allocation of 1D, 2D and 3D Array.

THEORY:

- Array is one of the linear data structure.
- Array is a collection of similar data type variables having contiguous (sequential) memory locations that share a common name.

ALGORITHM:

-

PROGRAMS:

- 1) Write a program to find duplicate elements in an array.
- 2) Given an array of integers, segregate even and odd numbers in the array. All the even numbers should be present first, and then the odd numbers.
- 3) Write an efficient program to find the sum of contiguous subarray within a one-dimensional array of numbers which has the largest sum.
- 4) Write a function rotate (arr[], d, n) that rotates arr[] of size n by d elements.

QUESTIONS:

- 1) Describe advantages and disadvantages of Array.
- 2) List out the applications of Array.

Department of
Information Technology

Week – 2, 3, 4 : Searching & Sorting

AIM:

- To implement different searching and sorting techniques.

OBJECTIVES:

- To understand basic concepts of searching and sorting.
- Able to compare different searching and sorting techniques.

THEORY:

Searching is an operation or a technique that helps find the place of a given element or value in the list. Any search is said to be successful or unsuccessful depending upon whether the element that is being searched is found or not. Some of the standard searching techniques that are being followed in the data structure are listed below:

- **Linear Search or Sequential Search :**
This is the simplest method for searching. In this technique of searching, the element to be found in searching the elements to be found is searched sequentially in the list.
- **Binary Search :**
Binary search is a very fast and efficient searching technique. It requires the list to be in sorted order. In this method, to search an element you can compare it with the present element at the center of the list. If it matches, then the search is successful; otherwise, the list is divided into two halves. The searching mechanism proceeds from either of the two halves depending upon whether the target element is greater or smaller than the central element. If the element is smaller than the central element, then searching is done in the first half; otherwise, searching is done in the second half.

Sorting refers to arranging data in a particular format. Sorting algorithm specifies the way to arrange data in a particular order. Most common orders are in numerical or lexicographical order. The importance of sorting lies in the fact that data searching can be optimized to a very high level, if data is stored in a sorted manner. Sorting is also used to represent data in more readable formats.

ALGORITHM:

Linear Search (A, N, X)

- 1) Start from the leftmost element of array A and one by one compare X with each element of array A.
- 2) If X matches with an element, return the index.
- 3) If X doesn't match with any of the elements, return -1.

Binary Search (A, N, X)

- 1) Compare X with the middle element of array.
- 2) If X matches with the middle element, we return the mid index.

- 3) Else If X is greater than the mid element, then X can only lie in right half subarray after the mid element. So we recur for right half.
- 4) Else (X is smaller) recur for the left half.

Bubble_Sort(K,N)

- 1) $LAST \leftarrow N$
- 2) Repeat thru step 5
for $PASS \leftarrow 1, 2, 3, \dots, N-1$
- 3) $EXCHS \leftarrow 0$
- 4) Repeat for $i \leftarrow 1, 2, \dots, LAST-1$
if $K[i] \geq K[i+1]$
then
 $K[i] \leftrightarrow K[i+1]$
 $EXCHS \leftarrow EXCHS + 1$
- 5) if $EXCHS = 0$
then
 Return
else
 $LAST \leftarrow LAST - 1$
- 6) Return

Selection_Sort(K,N)

- 1) Repeat thru step 4
for $PASS \leftarrow 1, 2, \dots, N-1$
- 2) $MIN_I \leftarrow PASS$
- 3) Repeat for $I \leftarrow PASS + 1, PASS + 2, \dots, N$
if $K[I] < K[MIN_I]$
then
 $MIN_I \leftarrow I$
- 4) if $MIN_I \neq PASS$
then
 $K[PASS] \leftrightarrow K[MIN_I]$
- 5) Return

Insertion Sort (A,N)

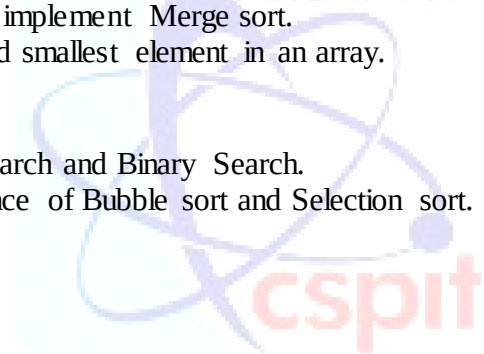
- for $j \leftarrow 2$ to n
- do $key \leftarrow A[j]$
 Insert $A[j]$ into the sorted sequence $A[1 \dots j-1]$
 $i \leftarrow j - 1$
 while $i > 0$ and $A[i] > key$
 do $A[i+1] \leftarrow A[i]$
 $i \leftarrow i - 1$
 $A[i+1] \leftarrow key$

PROGRAMS:

- 1) Let say you are a cashier in PQR Bank, people come to you when they want to withdraw money from their respective account. People have to come with their ADHAR CARD Photocopy. Prakash came and withdraw money from his account, of course by submitting adhar card photocopy, but on the second day he felt that submitted adhar card photocopy was not his, but adhar card no. he knew, which is (89891245).
 - a) How will you find that particular adhar card? (Suppose total 15 people withdrew their money on that day). Assume there is no fix pattern in all adhar cards.
 - b) How will you find that particular adhar card? (Suppose total 20 people withdrew their money on that day). Assume there is a pattern in all adhar cards, all are in ascending order (89891235 To 89891254). Develop this program using Iterative as well as Recursive approach.
- 2) Write a program to implement Bubble sort and Selection sort.
- 3) Write a program to implement Insertion sort to sort the cards in Playing Cards game.
- 4) Write a program to implement Merge sort.
- 5) Find Kth largest and smallest element in an array.

QUESTIONS:

- 1) Compare Linear Search and Binary Search.
- 2) Compare performance of Bubble sort and Selection sort.



Chandubhai S Patel
Institute of Technology

Department of
Information Technology

Week – 5, 6 : Stack

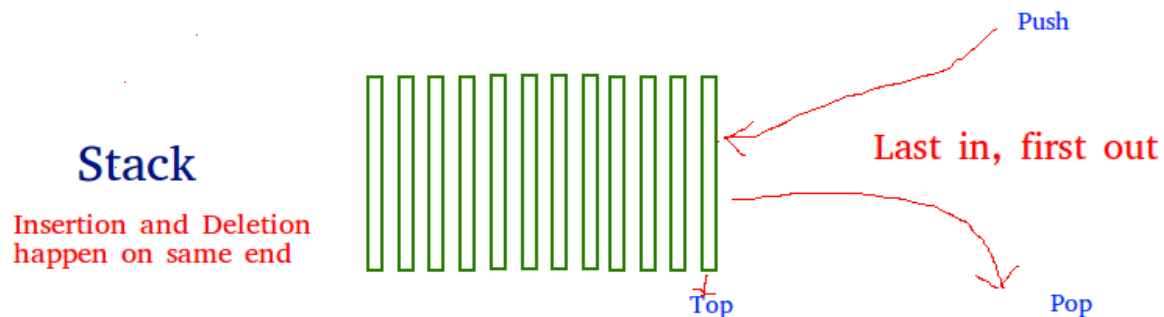
AIM: To implement Stack Data Structure.

OBJECTIVES:

- To understand basic functionality of Stack.
- To apply stack concepts to solve real life problem.

THEORY:

Stack is a linear data structure which follows a particular order in which the operations are performed. The order may be LIFO (Last In First Out) or FILO(First In Last Out).



Mainly the following three basic operations are performed in the stack:

- Push: Adds an item in the stack. If the stack is full, then it is said to be an Overflow condition.
- Pop: Removes an item from the stack. The items are popped in the reversed order in which they are pushed. If the stack is empty, then it is said to be an Underflow condition.
- Peek or Top: Returns top element of stack.
- isEmpty: Returns true if stack is empty, else false.

ALGORITHM:

PUSH (S, TOP, X): This procedure inserts an element X to the top of a stack which is represented by a vector S containing N elements with a pointer TOP denoting the top element in the stack.

- 1) [Check for stack overflow]
 - a. If $TOP \geq N$
 - b. then Write ("STACK OVERFLOW")
 - i. Return
- 2) [Increment TOP]
- 3) $TOP \leftarrow TOP + 1$
- 4) [Insert element]
- 5) $S[TOP] \leftarrow X$
- 6) 4. [Finished]

7) Return

POP (S, TOP): This function removes the top element from a stack which is represented by a vector S and returns this element. TOP is a pointer to the top element of the stack.

1) [Check for underflow on stack]

 If $TOP = 0$

 then Write ("STACK UNDERFLOW ON POP") take action in response to underflow

 Exit

2) [Decrement Pointer]

$TOP \leftarrow TOP - 1$

3) [Return former top element on stack]

Return (S [TOP + 1])

PEEP (S, TOP, I): Given a vector S (consisting N elements) representing a sequentially allocated stack, and a pointer TOP denoting the top element of the stack, this function returns the value of the ith element from the top of the stack. The element is not deleted by this function.

1) [Check for underflow on stack]

 If $TOP - I + 1 \leq 0$

 then Write ("STACK UNDERFLOW ON PEEP") take action in response to underflow

 Exit

2) [Return ith element from top of stack]

Return (S [TOP - I + 1])

□

PROGRAMS:

- 1) Write a menu driven program for Stack Operations using Array.(Push, Pop, Display)
- 2) Write a program that enters valid Infix expression and convert it into Postfix expression.
- 3) Write a program that enters valid Postfix/Prefix expression and evaluate the expression.
- 4) Write a program to print first n Fibonacci numbers using Recursion.
- 5) Write a program to implement Tower of Hanoi Puzzle using Recursion.

Tower of Hanoi is a mathematical puzzle where we have three stacks and n disks. The objective of the puzzle is to move the entire stack to another stack, obeying the following simple rules:

- 1) Only one disk can be moved at a time.
 - 2) Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack i.e. a disk can only be moved if it is the uppermost disk on a stack.
 - 3) No disk may be placed on top of a smaller disk.
- 6) Write a program to check balanced parentheses in an expression using stack. Code can contain any brackets from the set $[\{ \} ()]$.

QUESTIONS:

- 1) Differentiate Iterative Function and Recursive Function
- 2) List out the applications of Stack.
- 3) Why do we convert infix expression into postfix expression?

Week – 7, 8: Queue

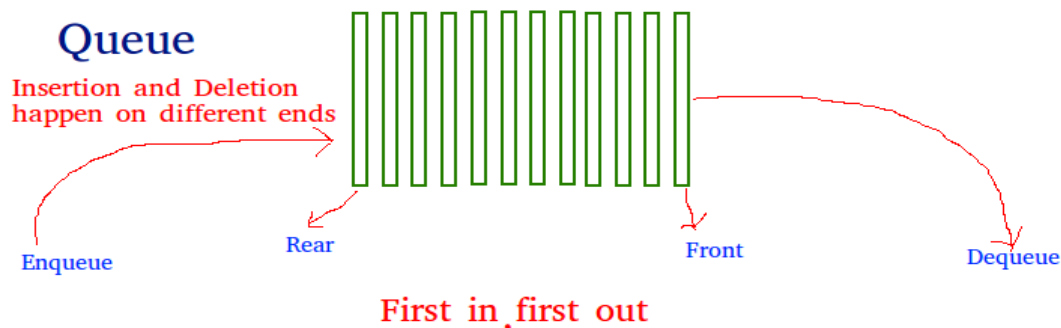
AIM: To implement simple Queue and Circular Queue.

OBJECTIVES:

- To understand working of different types of Queues.
- To select queue data structure to solve real life application.

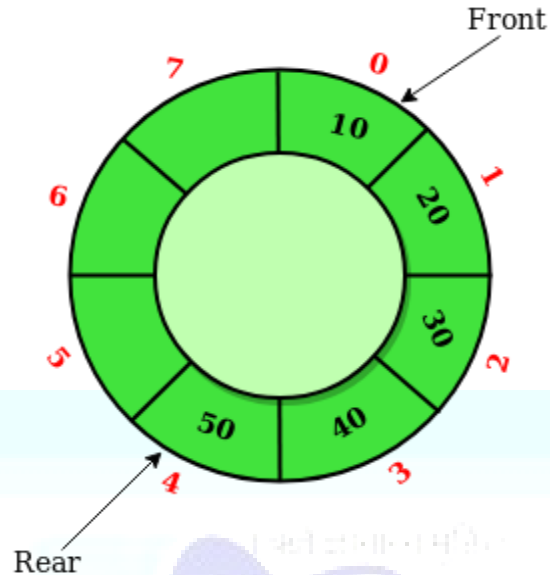
THEORY:

A Queue is a linear structure which follows a particular order in which the operations are performed. The order is First In First Out (FIFO). A good example of a queue is any queue of consumers for a resource where the consumer that came first is served first. The difference between stacks and queues is in removing. In a stack we remove the item the most recently added; in a queue, we remove the item the least recently added.



Circular Queue is a linear data structure in which the operations are performed based on FIFO (First In First Out) principle and the last position is connected back to the first position to make a circle. It is also called '**Ring Buffer**'.

Department of
Information Technology



ALGORITHM:

QINSERT (Q, F, R, N, Y). Given F and R, pointers to the front and rear elements of a queue, a queue Q consisting of N elements, and an element Y, this procedure inserts Y at the rear of the queue. Prior to the first invocation of the procedure, F and R have been set to zero.

1. [Overflow?]

If $R \geq N$

then Write ("OVERFLOW")

Return

2. [Increment rear pointer]

$R \leftarrow R + 1$

3. [Insert element]

$Q[R] \leftarrow Y$

4. [Is front pointer properly set?]

If $F=0$

then $F \leftarrow 1$

Return

QDELETE (Q, F, R). Given F and R, the pointers to the front and rear elements of a queue, respectively, and the queue Q to which they correspond, this function deletes and returns the last element of the queue. Y is a temporary variable.

1. [Underflow?]

If $F = 0$

then Write ("UNDERFLOW")

Return(0) (0 denotes an empty queue)

2. [Delete element]

$Y \leftarrow Q[F]$

3. [Queue empty?]

If $F = R$

then $F \leftarrow R \leftarrow 0$

else $F \leftarrow F + 1$ (Increment Front pointer)

4. [Return element]

Return (Y)

CQINSERT (F, R, Q, N, Y). Given pointers to the front and rear elements of a circular queue, F and R, a vector Q consisting of N elements, and an element Y, this procedure inserts Y at the rear of the queue. Initially, F and R are set to **zero**.

1. [Reset rear pointer?]

If $R = N$

then $R \leftarrow 1$

else $R \leftarrow R + 1$

2. [Overflow?]

If $F = R$

then Write ("OVERFLOW")

Return

3. [Insert element]

$Q[R] \leftarrow Y$

4. [Is front pointer properly set?]

If $F = 0$

then $F \leftarrow 1$

Return

CODELETE (F, R, Q, N). Given F and R, pointers to the front and rear elements of a circular queue, respectively, and a vector Q consisting of N elements, this function deletes and returns the last element of the queue. Y is a temporary variable.

1. [Underflow?]

If $F = 0$

then Write ("UNDERFLOW")

Return(0)

2. [Delete element]

$Y \leftarrow Q[F]$

3. [Queue empty?]

If $F = R$

then $F \leftarrow R \leftarrow 0$

Return (Y)

4. [Increment front pointer]

If $F = N$

then $F \leftarrow 1$

else $F \leftarrow F + 1$

Return (Y)

PROGRAMS:

- 1) We are developing software for a call center. When a client calls, his/her call should be stored until there is a free service representative to pick the call. Calls should be processed in the same order they are received. Select appropriate data structure to build call center software system.
- 2) Write a program to control Traffic Signal System. Switch on the traffic lights (Red, Yellow, and Green) one by one repeatedly as per the described time set.
- 3) Write a program to implement Stack using Queue.
- 4) Write a program to implement Queue using Stack.
- 5) Write a Program to implement Double Ended Queue operations. (insert, delete, display)

QUESTIONS:

- 1) Compare Simple Queue and Circular Queue.
- 2) Compare Stack and Queue Data Structure
- 3) List out applications of Queue.

Week – 9, 10: Linked List

AIM:

- To implement Linked list.

OBJECTIVES:

- To understand how linked list allocates memory.

THEORY:

- 5) A linked-list is a sequence of data structures which are connected together via links.
- 6) Linked List is a sequence of links which contains items. Each link contains a connection to another link. Linked list the second most used data structure after array. Following are important terms to understand the concepts of Linked List.

ALGORITHM:

I. Insertion

1. Ptr=List;
2. While (ptr<>NULL) repeat steps 3 to 4
3. If (DATA(ptr)='VAL') then
{
 X=new node
 Read (DATA(X);
 NEXT(X)=NEXT(ptr)
 NEXT(ptr)=X;
 BREAK;}
4. ptr=NEXT(ptr)
 [end of while]
5. END.

II. Searching and Deletion

- 1.If (DATA(list)='VAL')then
{
 Ptr=LIST;
 LIST=NEXT(list);

```

        Delete ptr;
        Stop;
    }
    Back=list;
    Ptr=list;
2.while(ptr<>NULL) repeat step 3 to 5
3.If(DATA(ptr)='VAL') then
{
    NEXT(back)=NEXT(ptr);
    Delete ptr;
    Exit;}
4.back=ptr;
5.ptr=next(ptr);
    [end of while loop]
6.END

```

PROGRAMS:

- 1) Write code for creating a linked list, adding elements to the linked list at the tail, searching for an element with a particular key and also deleting an element with a particular key if it exists.
- 2) Write a program to implement Undo and Redo functionality of Text Editor. (Doubly Linked List)
- 3) Write a program to implement singly linked list and find following
Count number of nodes, Find middle element, Reverse the singly linked list, get Nth node value
- 4) Write a program to implement dynamic Stack. (push, pop, display)
- 5) Write a program to implement dynamic Queue. (insert, delete, display)
- 6) Write a program to implement circular linked list. (insert, delete, display)

QUESTIONS:

- 1) List out Advantages and Disadvantages of Doubly linked list over Singly Linked List.
- 2) Compare Array and Linked List data structure.

Week – 11,12: Tree

AIM:

- To implement tree and its traversals.

OBJECTIVES:

- To understand basic concepts of tree implementation.

THEORY:

Tree represents the nodes connected by edges. We will discuss binary tree or binary search tree specifically. Binary Tree is a special data structure used for data storage purposes. A binary tree has a special condition that each node can have a maximum of two children. A binary tree has the benefits of both an ordered array and a linked list as search is as quick as in a sorted array and insertion or deletion operation are as fast as in linked list.

Traversal is a process to visit all the nodes of a tree and may print their values too. Because, all nodes are connected via edges (links) we always start from the root (head) node. That is, we cannot randomly access a node in a tree. There are three ways which we use to traverse a tree –

In-order Traversal

Pre-order Traversal

Post-order Traversal

ALGORITHM:

I. Inorder

Until all nodes are traversed –

Step 1 – Recursively traverse left subtree.

Step 2 – Visit root node.

Step 3 – Recursively traverse right subtree.

II. Preorder

Until all nodes are traversed –

Step 1 – Visit root node.

Step 2 – Recursively traverse left subtree.

Step 3 – Recursively traverse right subtree.

III. Postorder

Until all nodes are traversed –

Step 1 – Recursively traverse left subtree.

Step 2 – Recursively traverse right subtree.

Step 3 – Visit root node.

PROGRAMS:

- 1) Write a menu driven program to construct Binary Search Tree of your mobile phone number and apply different tree traversal techniques on BST. Find largest digit and smallest digit. Also search particular digit, If that digit is available in BST than prints its predecessor and successor.
- 2) Write a program to implement level order and zigzag Tree traversal techniques.
- 3) Write a program to construct AVL tree.

QUESTIONS:

- 1) Draw BST for given Preorder and In-order traversal
Preorder: 7,10,4,3,1,2,8,11 In-order: 4,10,3,1,7,11,8,2
- 2) List out applications of Tree.



Chandubhai S Patel
Institute of Technology

Department of
Information Technology

Week – 13: Graph

AIM:

- To implement graph traversals technique.

OBJECTIVES:

- To understand graph traversals technique thoroughly.

THEORY:

A graph is a pictorial representation of a set of objects where some pairs of objects are connected by links. The interconnected objects are represented by points termed as vertices, and the links that connect the vertices are called edges. Formally, a graph is a pair of sets (V, E) , where V is the set of vertices and E is the set of edges, connecting the pairs of vertices. Take a look at the following graph.

Depth First Search (DFS) algorithm traverses a graph in a depth ward motion and uses a stack to remember to get the next vertex to start a search, when a dead end occurs in any iteration.

Breadth First Search (BFS) algorithm traverses a graph in a breadth ward motion and uses a queue to remember to get the next vertex to start a search, when a dead end occurs in any iteration.

ALGORITHM:

I) DFS

- Step 1 – Visit the adjacent unvisited vertex. Mark it as visited. Display it. Push it in a stack.
- Step 2 – If no adjacent vertex is found, pop up a vertex from the stack. (It will pop up all the vertices from the stack, which do not have adjacent vertices.)
- Step 3 – Repeat step 1 and step 2 until the stack is empty.

II) BFS

- Step 1 – Visit the adjacent unvisited vertex. Mark it as visited. Display it.
Insert it in a queue.
- Step 2 – If no adjacent vertex is found, remove the first vertex from the queue.
- Step 3 – Repeat step 1 and step 2 until the queue is empty.

PROGRAMS:

- 1) Write a program to implement BFS Graph traversal technique.
- 2) Write a program to implement DFS Graph traversal technique.
- 3) Write a program to find whether given graph contains cycle or not.

QUESTIONS:

- 1) List out applications of Graph.
- 2) Compare BFS and DFS.



**Department of
Information Technology**

Week – 14: Hashing

AIM:

- To implement basic hashing technique and collision resolution technique.

OBJECTIVES:

- To understand hashing and its collision resolution techniques.

THEORY:

Hashing is a technique to convert a range of key values into a range of indexes of an array. We're going to use modulo operator to get a range of key values. Consider an example of hash table of size 20, and the following items are to be stored. Item are in the (key,value) format.

Following are the basic primary operations of a hash table.

Search – Searches an element in a hash table.

Insert – inserts an element in a hash table.

delete – Deletes an element from a hash table.

ALGORITHM:

--

PROGRAMS:

- 1) In an array of 20 elements, arrange 15 different values, which are generated randomly between 1,00,000 to 9,99,999. Use hash function to generate key and linear probing to avoid collision. $H(x) = (x \bmod 18) + 2$. Write a program to input and display the final values of array.
- 2) An array of 10 elements, and hash function is $(x \bmod 10)$. Handle Collisions using Separate Chaining. Keys: 75, 66, 42, 192, 91, 40, 49, 87, 67, 16, 417, 130, 372, 227. Write a program display the final values.

QUESTIONS:

- 1) What is hashing? Discuss the importance of Hashing.
- 2) List out applications of Hashing.